

- 1)
- Create a new node.
 - Enter the data.
 - Make the new node point to the current head
(new $\rightarrow$ next = head)
 - Update the head pointer (head = new).

Example:

Before:

head $\rightarrow$

2	
---	--

 $\rightarrow$

3	
---	--

 $\rightarrow$

4	
---	--

 $\rightarrow$ NULL

Head

newnode $\rightarrow$

1	
---	--

 $\rightarrow$ NULL

$\downarrow$

10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ NULL

Insert 5

After:

Head

$\downarrow$

5 $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ NULL

Logic:

1. Create a new node.
2. Store data
3. new $\rightarrow$ next = head
4. head = new

Time complexity: $O(1)$

To display the linked list

temp = head

while (temp $\neq$ NULL)

{

print (temp $\rightarrow$ data);

temp = temp $\rightarrow$ next;

}

to find the length:

count = 0;

temp = head;

while (temp != NULL)

{

count = ++;

temp = temp → next;

Example :

Linked list :

Head

↓

5 → 10 → 20 → 30 → NULL

Display output :

5 10 20 30

Length = 4

Logic :

1. Start from the head node.
2. Visit each node one by one.
3. Print the data.
4. Increase the count by 1 for each node.
5. Stop when temp = NULL

Time Complexity :

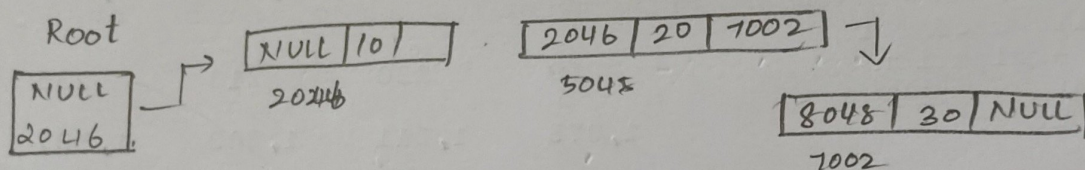
- Display : $O(n)$
- Find length : $O(n)$

3) Display Print the data.

```
void display()
{ struct node * temp = root;
  while (temp != null)
  { printf ("%d \n", temp->data);
    temp = temp->right;
  }
```

Algorithm:

1. Start from head node.
2. If head == NULL, Print "tree is empty",
3. Otherwise, move each node.
4. Stop when temp == NULL.



Traverse:-

```
temp -> 10 (Print 10)
temp -> 20 (Print 20)
temp -> 30 (Print 30)
temp -> NULL (stop).
```

Code:

```
void at begin()
{ struct node * temp;
  temp = (struct node *) malloc size of (struct)
  printf ("Enter node data");
  scanf ("%d", &temp->data);
  temp->left = NULL;
  temp->right = NULL;
  if (root == NULL)
  { root = temp;
  } else
  { temp->right = root
  }
```